

Combine

Composing with Algorithms
<http://www.bjarni-gunnarsson.net>

```

(
  NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFm1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
  }).play;
)

(
  NF(\dsc, {|freq = 1080|
    HPF.ar(
      BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
      ), 80)
    ).play;
)

var <>pindex, <>cindex;

initialize {
  if(pindex.isNil, { pindex = 1000 });
  if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
  pindex = 1000;
  (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
  if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
  if(index.isNil && pindex.isNil, {
    this.initialize();
  });

  pindex = pindex + 1;
  this[pindex] = \filter -> process;
}

control {|process, index|
  var i = index;

  if(i.isNil, {
    this.initialize();
    cindex = cindex + 1;
    i = cindex;
  });

  this[i] = \pset -> process;
}

(
  NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
      LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
  }).play;
)

```

Exercises

Exercises - Basics

1. Create a simple pattern using `Pbind` that plays a sine wave with random pitches using `Pwhite` for MIDI notes between 48 and 72.
2. Create a pattern that uses `Pgauss` (Gaussian distribution) to generate pitches centered around middle C with different standard deviations.
3. Implement a pattern using `Pbrown` (Brownian motion) where the pitch walks randomly within a range, with a step size of 2 semitones.
4. Create a pattern using `Pexprand` for both pitch and duration to create exponentially distributed random values.
5. Use `Pseg` to create a tendency mask where the pitch slowly evolves from low to high values over 8 seconds.

Exercises - FM

6. Create an FM synthesis pattern where the carrier-to-modulator ratio changes using `Pseq` through the values [1.5, 2, 3, 4].
7. Implement an FM pattern where both the modulation index and frequency are controlled by `Pexprand` to create varied timbres.
8. Create a pattern using the 4-operator FM synth (`\fmSine`) where each modulator frequency is controlled by a different probability distribution.
9. Build an FM texture where density increases over time using `Poisson` distribution for durations.

Exercises - Subtractive

10. Create a subtractive synthesis pattern using a sawtooth wave where the cutoff frequency sweeps from low to high using `Pseg`.
11. Implement a pattern that uses `Pbrown` to control both the pitch and filter resonance (rq) of a filtered sawtooth wave.
12. Create a dense cloud of filtered sawtooth events using `Pgauss` for pitch and `Pexprand` for filter cutoff.

Exercises - Patterns and Effects

13. Create a pattern that sends synthesized sounds to a reverb effect bus and experiment with different reverb decay times.
14. Implement a pattern with an echo effect where the echo length changes over time using a fork to modify the effect parameters.
15. Create a noise-based pattern that feeds into a dynamic resonator, where the resonator frequencies are updated periodically.
16. Build a sound that uses multiple effects in series (reverb → distortion) and experiment with the order of effects.

Exercises - Timbral

17. Create a granular texture using very short durations (< 0.1 seconds) and high density with filtered noise.
18. Implement a sound that uses chaotic noise (`Crackle`) with different chaos parameters controlled by `Pbrown`.
19. Create a resonant noise texture where multiple resonant frequencies are heard by using `Resonz` or the resonator effect.
20. Build a texture using `Dust` impulses sent through a comb filter or delay network to create bell-like timbres.
21. Combine multiple noise sources (white, pink, crackle) playing simultaneously through different effects to create a complex soundscape.